MapUnite — RoadPulse aur Rider Guardian: product aur technical blueprint

Brutally honest CPO / Android / ML / mobility-data review · October 2026

Pehle seedha jawab: **haan, MapUnite technically real hai** aur iska core (RoadPulse) aaj ki phone hardware pe ban sakta hai. Lekin kuch cheezein jo aap chahte ho, aaj honestly claim nahi ho sakti: "exact fuel", mic se wind speed, aur public launch ke din se hi "accurate pothole map".

Neeche har section mein yeh chaar hisse alag rakhe hain:

- **Ship now:** aaj ban sakta hai.
- **Validate next:** real riding data aur testing ke baad.
- **Future research:** possible hai, abhi worth nahi.
- **Never claim:** honestly bola hi nahi ja sakta.

Yeh review aapke repo ki current state ke against bhi likha hai, kyunki kuch cheezein pehle se bani hain aur kuch is spec se takraati hain.

---

1. Feasibility verdict

Repo mein jo pehle se bana hai (PR #20 tak):

- **Native Kotlin `DynamicsCore`:** 100 Hz IMU aur GNSS se jolt aur 20 m roughness nikalta hai. Jolt ki location **event ke timestamp** se nikalti hai, callback ke time se nahi. Dono ek hi clock (`elapsedRealtimeNanos`) par chalte hain, aur last GPS fix se speed + bearing laga ke position aage badhayi jaati hai (`positionAt`).
- **RoadSense log:** verification aur expiry ke rules.
- **Opt-in sync:** private zones aur daily pseudonym ke saath.
- **Route Truth Card, smooth passes, aur feed groundwork** (official lists aur predictions ke liye).
- **Fuel model ka base:** bike catalogue, aur fill-to-fill fuel calibration (km/L speed bands mein).

Is spec ke against jo gap ya conflict hai:

Gap / conflict	Kyun problem hai
Detector sirf **ek class "bump"** banata hai (7 m/s² threshold, ≥ 3 m/s)	Aapko 6 classes chahiye. Isliye chhoti unevenness bhi pothole ban jaati hai, yeh wahi problem hai jo aap dekh rahe ho.
Position **sirf pichhle fix** se dead-reckoning karti hai, aur speed newest fix ki leti hai	Agle fix ke saath interpolation nahi hota. 1 Hz GPS par 10–15 m error yahi se aata hai.
Abhi confirmation **automatic hai (sensor passes)**, explicit rider review nahi	Aapka "8 of 10 riders confirmed" explicit reviews par chalta hai. Dono alag signals hain, dono rakhne chahiye.
Card par **"confidence 91%"** dikhta hai	Yeh design priors se bana noisy-OR score hai, probability nahi. Aapke hi rule ke hisaab se yeh misleading hai, ise hatana ya rename karna chahiye.
Daily pseudonym	Ek rider do din alag ginta hai. Badges aur "unique riders" ke liye stable pseudonymous ID chahiye. Yeh privacy trade-off hai, section 13 mein hai.
Bump warning **2–8 s pehle** hoti hai	Speed-aware distance policy nahi hai. 60 km/h par 2 s bahut late hai.
Deep Search **Gemini/Groq keys** use karta hai	Aapke "no third-party AI API keys" rule se takraata hai. Local Ollama fallback pehle se hai; usi ko default banao, ya feature hatao.
Weather ke liye **Open-Meteo free API**	Uska free tier non-commercial use ke liye hai. Commercial launch se pehle paid plan lo ya IMD ka licensed access lo.

Hardware par seedhi baat:

- RTX 3050 (4 GB) par language model scratch se train nahi hoga. Sensor classification ke liye LLM ki zarurat bhi nahi hai.
- Aapke kaam ke liye classical ML aur ek chhota 1D-CNN kaafi hai. Dono yeh laptop seconds se minutes mein train kar dega.
- S24 FE par ek tiny model chalana trivial hai.

---

2. Rider value proposition (ek line)

"Jo rasta aap lene wale ho, us par haal hi mein riders ne asal mein kya mehsoos kiya: kahan gaddha hai, kahan breaker hai, kahan sadak theek thi, aur yeh baat kitne riders ne aur kab confirm ki."

Google aapko fastest route deta hai. MapUnite ko **road reality aur uska saboot** dena hai, honest uncertainty ke saath.

3. Build now / validate next / future research

Feature	Ship now	Validate next (real data chahiye)	Future research	Never claim
Event detection	Rolling buffer, event-time positioning, signal features, rule-based 6-class first cut	Classifier trained on labelled rides across bikes/mounts/phones; severity buckets	1D-CNN on raw windows; mount-invariant self-calibration	"Har pothole detect hota hai"
Post-ride review	Map of candidates, confirm/delete/edit/add, per-hazard publish	Review completion rate, time-to-review	Auto-suggested labels with active learning	—
Community agreement	Unique-rider counts, agreement %, freshness, disputed/repaired states	Calibration audit: confirmed hazards actually found?	Reputation-weighted alert thresholds	"80% chance pothole abhi hai"
Live alerts	Speed-aware distance, short calm voice, direction match	False-alert rate per 100 km, rider annoyance	Lane-level position (dual-frequency GNSS)	"Accident se bachata hai"
Badges	Wilson-bound accuracy badges, review credit, caps	Gaming attempts, fairness to rural riders	Regional "Ustaad" roles with moderation powers	—
Ride Story	Offline poster + 15 s video, masked start/end, preview/approve	Share rate, template preference	AI scenery detection from rider photos	—
Fuel estimate	Physics + fill-to-fill calibration (already partly built), uncertainty range	Per-bike CdA/Crr fit across many riders	Mic-based wind; OBD fuel rate where available	"Exact fuel burned"
Speed limits	OSM `maxspeed` where tagged + "limit unknown"	Coverage audit in your launch region	State-notified class-wise limits encoded per state	"Complete speed-limit data"
Speed cameras	OSM `highway=speed_camera` as "fixed camera reported"	Freshness & accuracy audit	—	"Sab cameras", "challan se bachao"
Number plate / VIN	Plate optional (emergency card only, on-device)	—	Manufacturer recall integration (with partner)	VAHAN access without authorisation

4. RoadPulse architecture

```
PHONE (offline-first)
 SensorHub (100 Hz accel+gyro+rotation, GNSS 1 Hz, same elapsedRealtimeNanos clock)
   └ RingBuffer 15 s (IMU) + GNSS fix list (last 30 s)
       └ CandidateDetector (cheap trigger, high recall)
           └ EventWindow extractor (−1.5 s … +1.5 s around peak)
               ├ Localiser: position at t_peak = interpolate(fix_before, fix_after) → snap to road
               └ Classifier (rules v0 → GBDT v1 → 1D-CNN v2): 6 classes + score
   └ RideStore (Room/SQLite, private): candidates, segments, smooth passes, trace (on-device only)
 Post-ride Review UI → user decisions → PublishQueue (only selected hazards)
 Live: HazardCache (offline tiles of public hazards along route) → AlertPolicy → TTS

SERVER (self-hosted)
 API (rate-limited) → Report ingest → snap to road segment+direction → dedupe/merge
   → hazard_reviews → hazard_stats (agreement, freshness, state) → tile export (route corridor)
 Moderation + abuse detection; contributor reputation; open-data ingest (versioned)
```

Do pipelines alag hain, aur alag rehne chahiye:

- **Sensor evidence:** phone ne automatically kya feel kiya.
- **Rider testimony:** rider ne review mein kya bola.

Card par dono dikhaao, lekin agreement % sirf testimony se banta hai.

5. Detection and consensus logic

5a. Event-time localisation (marker-late problem ka asli fix)

- **Ek hi clock:**
  - `SensorEvent.timestamp` aur `Location.getElapsedRealtimeNanos()` dono boot-time clock par hote hain.
  - Har test phone par dono log karke verify karo. Kuch purane devices par sensor timestamp alag base par aata hai.
- **Peak ka time lo, threshold crossing ka nahi:**
  - Abhi `DynamicsCore` jolt ka *start* (threshold crossing) leta hai.
  - Better hai buffer mein window lekar asli peak, ya pehle downward excursion ka time `t_e` nikaalna.
- **Do fixes ke beech interpolate karo:**
  - `t_e` ke pehle ka fix A aur baad ka fix B lo.
  - Agar B abhi aaya nahi, event ko 1 s "pending" rakho.
  - Phir B ke saath interpolate karo:

```
α = (t_e − t_A) / (t_B − t_A)
pos(t_e) = slerp-on-path(A, B, α)        // short gaps: linear lat/lng is fine
// agar B nahi aaya (tunnel/drop) → dead-reckon from A with speed & bearing (current behaviour)
```

- **Phone ka offset:** phone handlebar par hai, matlab lagbhag front axle ke upar. Front wheel ka hit hi asli location hai, isliye extra offset nahi lagana.
- **Snap to road:** point + heading ko road segment par project karo. Direction way ki geometry se aati hai.
  - **Server par:** sirf point + heading bhejo, poora trace nahi. Server apne OSM-derived road graph par snap karega.
  - **On-device (offline):** same algorithm offline road tiles par chalao.
  - Do candidate ways ho, jaise flyover ke neeche aur upar, toh heading aur elevation se tie-break karo. Phir bhi doubt ho toh "unknown" mark karo.
- **Error budget, ek misaal:** 1 Hz GNSS ka ~5 m error, 60 km/h (16.7 m/s) par 100 ms latency yaani ~1.7 m, aur event-timing ka 50 ms yaani ~0.8 m.
  - Asal error har phone par field-test karke napo, jaise surveyed potholes par 10 passes karke.
  - Phir median aur p90 error report karo.

5b. Classification: pehla practical approach

Laptop aur data ke liye best first model: signal-processing features + gradient-boosted trees (LightGBM/XGBoost), ya ek chhota random forest.

Kyun:

- Shuru mein aapke paas shayad kuch hazaar labelled events hi honge. Itne data par trees CNN se better generalise karte hain.
- Explain karna aasan hai, aur training seconds mein ho jaati hai.

Phone par kaise chalayein:

- `m2cgen` model ko plain Java/Kotlin code mein export kar deta hai. Isme koi runtime library nahi chahiye.
- Ya ONNX Runtime Mobile use karo.

Features, event window ±1.5 s, sab road frame mein:

- **Vertical axis:** gravity ka direction rotation vector se lo aur vertical acceleration nikaalo. Isse mount ka angle matter nahi karta.
- **Vertical signal:** peak up, peak down, sign order (pehle neeche gaya ya upar), duration, aur 0.5–5 Hz / 5–15 Hz / 15–30 Hz band energy.
- **Speed se normalise:**
  - peak ÷ speed;
  - vertical velocity change, yaani acceleration ka integral, lagbhag Δv_z;
  - jerk.
- **Gyro pitch rate:** breaker par bike ka nose upar-neeche hota hai.
- **Wheelbase double-hit:** pehle front wheel lagta hai, phir rear. Dono hits ke beech gap `Δt ≈ wheelbase / v` hota hai.
  - Wheelbase bike catalogue se lo (agar field hai).
  - Example: 1.3 m wheelbase, 30 km/h par $\Delta t \approx 0.16$ s.
  - Pothole aur breaker dono mein double hit hota hai. Random vibration mein nahi hota.
  - **Yeh feature validate karna padega**, kyunki handlebar par rear hit kamzor aata hai.

- **Shape:**
  - pothole: pehle neeche gir, phir upar jhatka;
  - breaker: pehle upar, phir neeche, lambi duration;
  - rough road: lamba aur sustained RMS, koi ek peak nahi.
- **Ride baseline:** ride ke pehle smooth minutes mein engine aur mount ki vibration ka baseline seekho, aur har feature ko us baseline ke z-score mein do. Har bike aur mount alag vibrate karta hai.
- **Context:** speed, GPS accuracy, mount-moved flag (SensorHub mein pehle se hai), aur braking event paas hai ya nahi. Hard braking ka pitch bhi breaker jaisa dikhta hai.

Classes aur unka asli matlab:

Class	Asli signal	Kyun mushkil
Severe pothole	Down-then-up, bada Δv_z, double hit	Bike ki suspension aur speed ke saath badalta hai
Small pothole	Wahi shape, chhota amplitude	Rough road aur expansion joint se confusion
Road hump / speed breaker	Up-then-down, lambi duration, pitch, rider pehle se slow	Rider slow karta hai, isliye signal chhota aata hai. Breaker se pehle ki braking bhi ek clue hai.
Rough-road segment	≥ 20–50 m tak sustained RMS	Chhota rough patch pothole jaisa dikh sakta hai
Normal vibration / non-road	Mount ka khadakna, kerb, rider ka haath, bike stand	Mount-moved aur speed < 3 m/s wale events reject karo
Unknown candidate	Classifier ka score kam	Ise kabhi public alert mat banao. Sirf review mein dikhao.

Model upgrade path:

- **v0 (abhi):** existing threshold + shape rules + speed normalisation.
- **v1 (~2–5k labelled events):** GBDT. Train/test split **rider, bike aur phone ke hisaab se group karo** (group k-fold).
  - Random split mat karo: ek hi ride ke events dono taraf aa jaate hain aur accuracy jhoothi badh jaati hai.
- **v2 (~20k+ events, kai bikes):**
  - Model: 1D-CNN; input 3 s window, 100 Hz, 6–8 channels (road-frame accel + gyro + speed).
  - Size: ~50–200k parameters.
  - Training: RTX 3050 par minutes mein.
  - Export: int8 TFLite (LiteRT) ya ONNX.
  - Ship tabhi karo jab group-split test par v1 ko hara de.
- **GGUF:** sirf llama.cpp ke LLM ke liye hai. Sensor model ke liye irrelevant.
- **Chhota LLM:** agar kabhi chahiye (backend par Q&A phrasing ke liye), toh 1–3B model Q4 GGUF mein. QLoRA fine-tune aapke laptop par slowly ho sakta hai. Lekin road truth fixed rules aur phrases hi bolenge.

5c. Labelled data: data collection jo sach mein kaam kare

1. **Ground-truth rides (aap aur aapki team):**
   - Ek second phone ya action cam se road record karo. `DatasetRecorder` pehle se 2 fps frames + `imu.csv` + `gnss.csv` likhta hai.
   - Pothole/breaker ka label video dekh ke do. Yeh sabse saaf label hai.
2. **Rider reviews (weak labels):**
   - Confirm/delete/edit ek label hai, lekin noisy hai.
   - Inhe model ki training mein lower weight do, aur ground-truth wale set par validate karo.
3. **Coverage matrix track karo:**
   - bike class (commuter / sports / cruiser / scooter);
   - suspension;
   - mount type (rigid, vibration-damped, tank bag);
   - phone model;
   - speed band;
   - road type;
   - weather (geeli sadak).

   Jis cell mein data nahi, us combination ke liye accuracy claim mat karo. 4. **Labelling tool:** ek chhota local web tool (laptop par): - signal plot, map point aur video frame ek saath dikhe; - keyboard shortcuts 1–6 se class lag jaaye.

5d. Community agreement (aapka formula, kuch guards ke saath)

```
Window W(type): e.g. pothole 30 din, hump 365 din, rough 90 din, waterlogging 12 ghante   (design priors, tune karo)
Votes = har unique rider ka sirf LATEST review is hazard par, W ke andar,
        aur sirf "last repaired event" ke baad ke
C = "I found it" unique riders
N = "I did not find it" unique riders   (sirf tab gine jab rider ka trace waqai wahan se, same direction mein, guzra)
U = "unsure" (count dikhao, formula mein nahi)

agreement = C / (C + N)        // dikhao sirf jab C + N ≥ 3; warna counts dikhao ("1 rider ne kal report kiya")
```

Display:

- "8 of 10 reviewing riders confirmed (last 30 days)"
- "Last confirmed 3 hours ago"
- Saath mein label: "Agreement among riders who reviewed. Not a measurement."

Inflation rokne ke rules:

- **Ek rider, ek vote:** har rider ka sirf latest review count hota hai. Same rider ka 10 baar guzarna 1 hi vote hai; repeat confirm sirf "last confirmed" time update karta hai.
- **Passage proof:** "Not found" aur "found" tabhi count hote hain jab:
  - phone ke private trace ne us jagah ko ±8 m mein cross kiya (prior);
  - same direction mein;
  - speed > 10 km/h;
  - GPS accuracy < 10 m.

  Phone sirf ek signed "passed: yes" summary bhejta hai, trace nahi. **Honest limit:** modified app is check ko fake kar sakti hai, isliye section 13 ki abuse defences zaroori hain. - **Reporter ki apni vote:** jo hazard rider ne khud report kiya, us par uska vote C mein ek hi baar ginta hai.

Merging aur direction:

- **Kaun se reports ek hazard hain:**
  - same road segment;
  - same direction (ya undivided road par "both");
  - same class family;
  - along-road distance ≤ max(12 m, 2σ_gps).

  Aisi reports ek hazard mein merge karo. Position sab reports ka GPS-accuracy-weighted median ho. - **Divided highway:** dono carriageways alag OSM ways hote hain, isliye direction automatically alag hoti hai. - **Undivided road:** pothole aksar ek hi lane mein hota hai. Direction "both" rakho jab tak dono taraf se reports na aayein. - **Severity:** reviewers ka mode/median dikhao. Disagreement ho toh "severity reports differ" likho.

State machine:

State	Rule (priors, tune karo)	Alert?
reported	1 rider, abhi review nahi	Map par haan; voice sirf tab jab severe ho, < 72 h purana ho, aur reporter ka trust ≥ baseline ho. Wording: "reported".
confirmed	≥ 2 unique "found", agreement ≥ 70%	Haan
disputed	C+N ≥ 3 aur agreement 35–65%, ya last 2 votes aapas mein ulte	Screen par; voice mein "recent reports disagree" ya chup
possibly repaired	Last confirm ke baad ≥ 2 unique "not found", koi naya confirm nahi	Voice nahi, map par grey
archived	≥ 3 "not found", ya W se purana	Nahi

Sensor evidence ko alag rakho:

- Automatic jolt passes se "Felt by 5 riders' sensors" dikhao. Repo ka current verification logic yahi karta hai.
- Agreement % ko yeh mat badhane do.
- Rider "I didn't find it" bole lekin uske sensor ne jolt feel kiya ho, toh dono record karo. Yahi accha training data hai.

---

6. Database schema (PostgreSQL + PostGIS)

Pilot ke liye current SQLite theek hai. Scale par PostGIS lo, kyunki spatial index aur segment queries wahan native hain.

```
-- road graph (derived from OSM; ODbL obligations apply — see §12)
CREATE TABLE road_segments (
  id            BIGSERIAL PRIMARY KEY,
  osm_way_id    BIGINT NOT NULL,
  seq           INT NOT NULL,                 -- ~50 m pieces along the way
  geom          geometry(LineString, 4326) NOT NULL,
  oneway        BOOLEAN NOT NULL,
  highway_class TEXT,
  maxspeed_kmh  INT,                          -- only if tagged; NULL = unknown
  source_version TEXT NOT NULL,               -- e.g. geofabrik-india-2026-10-15
  UNIQUE (osm_way_id, seq, source_version)
);

CREATE TABLE contributors (
  id            UUID PRIMARY KEY,             -- random; never shown publicly
  nickname      TEXT,                         -- optional, public only on profile
  created_at    TIMESTAMPTZ NOT NULL,
  trust_alpha   REAL NOT NULL DEFAULT 2,      -- Beta prior (see §9)
  trust_beta    REAL NOT NULL DEFAULT 2,
  status        TEXT NOT NULL DEFAULT 'active' -- active | limited | banned
);

CREATE TABLE hazards (
  id            UUID PRIMARY KEY,
  segment_id    BIGINT REFERENCES road_segments(id),
  offset_m      REAL NOT NULL,                -- along the segment
  direction     TEXT NOT NULL,                -- forward | backward | both | unknown
  geom          geometry(Point, 4326) NOT NULL,
  pos_sigma_m   REAL NOT NULL,
  class         TEXT NOT NULL,                -- severe_pothole | small_pothole | hump | rough | ...
  length_m      REAL,                         -- for rough/roadworks segments
  state         TEXT NOT NULL,                -- reported | confirmed | disputed | possibly_repaired | archived
  first_reported_at TIMESTAMPTZ NOT NULL,
  last_confirmed_at TIMESTAMPTZ,
  last_repaired_mark_at TIMESTAMPTZ,
  created_by    UUID REFERENCES contributors(id)  -- internal only
);
CREATE INDEX ON hazards USING GIST (geom);

CREATE TABLE hazard_reports (                 -- a rider published a candidate (from review)
  id            UUID PRIMARY KEY,
  hazard_id     UUID REFERENCES hazards(id),
  contributor_id UUID REFERENCES contributors(id),
  class         TEXT NOT NULL,
  severity      SMALLINT,                     -- 1..3 as rider chose
  geom          geometry(Point, 4326) NOT NULL,
  pos_sigma_m   REAL NOT NULL,
  heading_deg   REAL,
  event_time    TIMESTAMPTZ NOT NULL,         -- shown publicly only as "x hours ago"
  origin        TEXT NOT NULL,                -- sensor_confirmed | manual_add | edited
  model_version TEXT,
  model_score   REAL
);

CREATE TABLE hazard_reviews (                 -- found / not_found / unsure
  hazard_id     UUID REFERENCES hazards(id),
  contributor_id UUID REFERENCES contributors(id),
  verdict       TEXT NOT NULL,
  severity      SMALLINT,
  passed_attested BOOLEAN NOT NULL,           -- device says trace crossed it
  reviewed_at   TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (hazard_id, contributor_id, reviewed_at)
);
```

```
CREATE TABLE sensor_passes (                  -- automatic evidence, separate from testimony
  hazard_id     UUID REFERENCES hazards(id),
  contributor_hash TEXT NOT NULL,             -- per-day label is fine here
  felt          BOOLEAN NOT NULL,
  peak_norm     REAL,
  hour_bucket   TIMESTAMPTZ NOT NULL
);

CREATE TABLE segment_coverage (               -- "ridden, nothing felt" (smooth passes)
  segment_id    BIGINT REFERENCES road_segments(id),
  direction     TEXT NOT NULL,
  pass_label    TEXT NOT NULL,                -- unlinkable per-segment token hash
  hour_bucket   TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (segment_id, direction, pass_label)
);

CREATE MATERIALIZED VIEW hazard_stats AS ... -- C, N, U, agreement, last_confirmed, state

CREATE TABLE contributor_outcomes (           -- for reputation (§9)
  contributor_id UUID, hazard_id UUID, outcome TEXT, -- confirmed_by_others | refuted | unresolved
  resolved_at TIMESTAMPTZ
);
CREATE TABLE badges (contributor_id UUID, badge TEXT, region TEXT, awarded_at TIMESTAMPTZ,
                     PRIMARY KEY (contributor_id, badge, region));
CREATE TABLE moderation_flags (id UUID PRIMARY KEY, target_type TEXT, target_id UUID,
                               reason TEXT, created_at TIMESTAMPTZ, resolved TEXT);
CREATE TABLE data_sources (id TEXT PRIMARY KEY, licence TEXT, attribution TEXT, url TEXT,
                           fetched_at TIMESTAMPTZ, version TEXT, commercial_ok BOOLEAN, notes TEXT);
CREATE TABLE api_keys (id UUID PRIMARY KEY, owner TEXT, hash TEXT, plan TEXT,
                       created_at TIMESTAMPTZ, revoked_at TIMESTAMPTZ);
CREATE TABLE api_usage (key_id UUID, day DATE, calls INT, PRIMARY KEY (key_id, day));
```

Ride traces server par kabhi nahi jaate. `rides` aur `ride_points` sirf phone ke Room DB mein rehte hain.

---

7. Android sensor and offline ML design

Service setup:

- Ride shuru → **foreground service**, `foregroundServiceType="location"`. Mic feature ho toh `|microphone` bhi.
- Android 14+ par `FOREGROUND_SERVICE_LOCATION` aur `FOREGROUND_SERVICE_MICROPHONE` permissions chahiye.
- Notification hamesha dikhe, uspe "Stop ride" ho.

Sensors:

- `TYPE_LINEAR_ACCELERATION` ya raw accel + `TYPE_ROTATION_VECTOR` + gyro, 10,000 µs (100 Hz) par.
- 200 Hz se upar ke liye Android 12+ par `HIGH_SAMPLING_RATE_SENSORS` permission chahiye. Zarurat nahi, 100 Hz kaafi hai.
- Batching (`maxReportLatencyUs`) se battery bachti hai. Timestamp har sample ka asli hota hai, isliye event-time positioning safe rehti hai.

GNSS:

- Fused provider, 1 Hz, high accuracy.
- Har fix ka `elapsedRealtimeNanos`, accuracy, bearing aur speed store karo.

Ring buffer:

- Float arrays par fixed size circular buffer (15 s × 100 Hz × ~10 channels ≈ 15k floats). Allocation-free, GC jitter nahi.
- Detector sirf candidate trigger karta hai. Classifier background thread par chalta hai. Location event time se aati hai, isliye classifier late chale toh koi farak nahi.

Model runtime:

- **v1 (GBDT):** m2cgen se Kotlin/Java code, ya ONNX Runtime Mobile. Sub-millisecond per event.
- **v2 (1D-CNN):** LiteRT (TFLite) int8, CPU par. NNAPI/GPU ki zarurat nahi, model tiny hai.

Model delivery:

- Model file versioned ho (`model_version` har report mein jaaye).
- Naya model app update se aaye, ya signed download ke through.

- Rollback possible ho.

Battery aur thermal:

- Field mein napo: %/hour, S24 FE aur ek budget phone (jaise 4 GB RAM) dono par.
- Handlebar par dhoop mein phone garam hota hai. Existing `ThermalMonitor` ka use karo.

Offline hazard cache:

- Route set hote hi us corridor ke hazards (tile-wise) download karo. Repo ka sync yahi karta hai.
- Bina signal ke bhi alerts chalne chahiye.

---

8. Post-ride review UX and live alerts

Post-ride review (target: 30 second se kam)

1. **Ride khatam hone ke baad, bike rukne par:**
   - Ek card: "Is ride mein 4 jagah jhatke mile. 30 sec mein check karein?" Buttons: [Abhi] [Baad mein] [Kabhi nahi].
   - Ride ke dauraan kabhi mat poochho.
2. **Map + bottom carousel:** har candidate ek card hai. Card par:
   - map pin, aur pin ke liye ±σ circle;
   - class ka guess;
   - "8.4 km pe, 42 km/h par";
   - ek chhota signal sparkline.
3. **Har card par 3 bade buttons:**
   - **[Haan, gaddha tha]** (severity chips: chhota / bada)
   - **[Breaker tha]**
   - **[Kuch nahi tha]**

   Swipe left = delete, swipe right = confirm. Long-press karke pin drag karo (location edit). 4. **"+ Gaddha jodo":** map par tap karke missed hazard add karo. "Rough stretch" ke liye do points tap karo. 5. **Publish screen:** - Default: sirf confirmed hazards selected hon, unknown wale nahi. - Har hazard ke saath toggle: "Public road report banayein". - Saaf likho kya jayega: "Sirf yeh point, direction, type, severity, aur 'x ghante pehle'. Aapka rasta, ghar, naam nahi." 6. **Speed tricks:** - Agar review mein rider ka verdict model ke guess se match kare, toh agle ride se us class ke high-score candidates pre-selected aayein. - 0 candidates → review screen skip. - **Unknown candidates kabhi auto-publish nahi.**

Doosre rider ke liye review prompt (Day 2 example)

Ride ke baad: "Aapke raaste pe 2 reported hazards the. Kya mile?" Har ek par [Mila] [Nahi mila] [Pata nahi].

- Sirf woh hazards dikhao jahan se aap sach mein guzre (passage check).

Live alert policy (speed-aware)

```
v      = current speed (m/s), smoothed
v_t    = target speed at hazard: severe/hump ~20 km/h, small/rough ~30 km/h   (priors)
t_say  = time to speak the line (~2.5 s for a short line)
t_r    = reaction time (AASHTO stopping-sight design uses 2.5 s perception-reaction;
         use ≥ 1.5 s for an expected cue)
a      = gentle deceleration, ~1.5 m/s² (comfort prior; AASHTO's 3.4 m/s² is for stopping, too harsh)
σ      = 2 × GPS accuracy + hazard pos_sigma

d_alert = v·(t_say + t_r) + max(0, v² − v_t²)/(2a) + σ
```

Misaal:

Speed	Target	Calculation	d_alert
60 km/h (16.7 m/s)	20 km/h	16.7×4 = 67 m, plus (279−31)/3 = 83 m, plus σ ~15 m	**~165 m**
20 km/h (5.6 m/s)	15 km/h	5.6×4 = 22 m, plus ~5 m, plus σ ~15 m	**~42 m**

Kab bolna hai:

- Sirf tab jab hazard ki direction rider ki heading se match kare (ya "both" ho), aur rider route ke us segment par ho (map-matched).

- Alert sirf `d_alert ± v·1 s` window mein bolo. Agar rider pehle se us distance ke andar hai, toh sirf tab bolo jab kam se kam 2 s bachein. Warna chup raho, late voice distraction hai.

Kya bolna hai:

- Ek line, calm: "Aage road hazard report hua hai. Dheere-dheere speed kam karein." / "Reported road hazard ahead. Slow down gradually and watch the road."
- Severity se wording badlo, volume nahi.
- **"Brake", "abhi ruko", ya beep-storm kabhi nahi.**

Clustering aur priority:

- 500 m mein 3+ hazards → ek line: "Agle 500 metre mein kai gaddhe reported hain."
- Turn instruction ke 4 s andar hazard alert mat do. Turn pehle bolo, hazard baad mein, warna screen par.

Kis hazard par alert nahi:

- Disputed, possibly repaired aur unknown par voice nahi.
- Reported (1 rider) par voice sirf section 5d ki condition par.

Details: rider tap karke details tabhi dekhe jab ruka ho (speed < 5 km/h). Wahan count, agreement, last confirmed, severity aur badge level dikhao.

---

9. Badge and reputation design

Accuracy kaise napo:

```
For each published report, outcome resolves after ≥ 3 independent reviews (other riders):
  confirmed_by_others → success; refuted (agreement < 35%) → failure; else unresolved (no effect)
trust = Beta(α0 + successes, β0 + failures), α0 = β0 = 2   // starts neutral; unresolved never punishes
Badge gates use the Wilson lower bound of success rate (95%), not the raw ratio:
  Road Scout:          ≥ 5 resolved, LB ≥ 0.5
  Trusted Road Scout:  ≥ 20 resolved, LB ≥ 0.7
  Road Guardian:       ≥ 60 resolved, LB ≥ 0.8, active in last 90 days
  Local Route Expert:  ≥ 25 resolved in one district + ≥ 50 reviews there
```

Yeh sab thresholds priors hain. Pilot mein dekhna hai ki kitne riders sach mein inhe paate hain.

Points ke rules:

- **Kis cheez ke points:**
  - sirf independently confirmed reports;
  - honest reviews (review karna bhi contribution hai, "not found" bhi).
- **Kis cheez ke points nahi:**
  - raw report count;
  - kilometres;
  - speed;
  - streaks jo roz ride karne ka pressure banayein.
- **Caps:** daily point cap, aur ek hazard par ek hi baar credit.
- **Likes / "helpful":** alag counter, sirf "community thanks" dikhane ke liye. Accuracy mein inka koi role nahi.
- **Uncertain reports:** "unsure" aur unresolved reports se trust kabhi nahi girta.

Leaderboards:

- Opt-in, district-level, weekly.
- Rank "confirmed contributions" se ho, report volume se nahi.
- Ride ke dauraan kabhi nahi dikhe. Koi "is hafte sabse zyada km" leaderboard nahi.

Badge naam jo Indian riders ko pasand aayenge:

- **Rasta Scout** (Road Scout)
- **Pakka Scout** (Trusted Road Scout)
- **Sadak Prahari** (Road Guardian; prahari = rakhwala)
- **Ilaake ka Ustaad** (Local Route Expert), region ke naam ke saath, jaise "Koraput ka Ustaad"
- Seasonal: **Monsoon Marshal**, yaani baarish mein waterlogging aur gaddhe confirm karne wale. Iske liye baarish mein ride karne par koi bonus nahi, sirf confirmed reports par.
- Hills ke liye: **Ghat Guardian**.

- Fun, low-key: **Gaddha Hunter** (sirf profile par).

---

10. Ride Story design

Offline implementation:

- Ride summary JSON se rendering:
  - Canvas/Compose frames, ya Jetpack Media3 **Transformer** compositions (offline, on-device);
  - ya seedha `MediaCodec` (H.264) + `MediaMuxer`.
- Format: 1080×1920, 30 fps, 10–15 s.
- **Map background:** offline OSM vector data se stylised render. Frame mein "© OpenStreetMap contributors" zaroor ho.
- **Music:** copyrighted music bundle mat karo. Ya toh royalty-free tracks jinke commercial licence aapke paas hon, ya bina music.
- **Start/end masking (default on):**
  - pehla aur aakhri 1 km route line se kaat do (existing private zones bhi use karo);
  - wahan ke labels mat dikhao;
  - sirf city/district naam dikhao.
- **Preview → approve → share** (Android share sheet). Bina approval kuch share nahi hota.

Templates:

Template	Kya dikhata hai
Route Reveal	Animated line + distance, time, elevation gain
Ghat Climb	Elevation profile highlight
Monsoon Run	Weather badge + "ride safe" tone
Rasta Scout Report	"Aaj 3 hazards map kiye, 12 km smooth verify ki". Contribution focus, sabse strong brand angle.
Static poster	Rider ki photo + stats card

Kya mat karo:

- Top speed default off rakho. Rider khud on kare tabhi dikhe, aur bina "fastest" language ke.
- Average speed ko hero mat banao.
- "Personal best time" type cards mat banao.

Fuel line: sirf "Est. fuel ~1.8–2.3 L" format mein, saath mein "estimate" label.

---

11. Fuel and wind estimation (honest version)

Physics se abhi kya calculate ho sakta hai (post-ride, ek baar):

```
F_roll  = Crr · m · g · cos θ                    // m = bike + rider + pillion + luggage → yahan weight matter karta hai
F_grade = m · g · sin θ                          // elevation/slope
F_inert = m · a  (only positive work counts; braking energy is lost on an ICE bike)
F_aero  = ½ · ρ · CdA · |v_rel| · v_rel,  v_rel = v_ground − w_along   // wind along heading
ρ       = p / (R_d · T)                          // from temperature, pressure/elevation
P_wheel = (F_roll + F_grade + F_inert + F_aero) · v
fuel    = Σ P_wheel / (η_drivetrain) · BSFC(load, rpm) · Δt / ρ_fuel
```

Weight aur drag ka farak:

- **Weight** rolling resistance, chadhai aur acceleration ko badalta hai.
- **Drag** weight se nahi, **relative wind aur bike+rider ke frontal shape (CdA)** se badalta hai.
- Isliye pillion ke saath highway par extra fuel ka bada hissa chadhai aur acceleration se aata hai, aur CdA bhi thoda badhta hai.

Calibration kyun zaroori hai:

- CdA, Crr, η aur BSFC curve bike-wise alag hote hain. Inki published values bhi bahut bikhri hui hain, isliye inhe guess nahi, **fit** karna padega.
- Repo mein pehle se fill-to-fill learning hai: real km/L speed bands mein, aur generic curve tabhi replace hota hai jab woh behtar predict kare. Yahi sahi base hai.
- **Aage:** har bike model ke liye kai riders ke fill-to-fill data se effective CdA aur Crr ka Bayesian fit karo. Har rider ka personal multiplier usi ke data se.

3D geometry ya CFD kyun nahi:

- Bike ka 3D model rider ka posture, kapde, luggage, windscreen aur mirrors miss karta hai, aur drag ka bada hissa wahi hai.
- Har ride ke baad CFD mehenga aur bekaar hai.
- Practical tareeka: **ek baar ka calibrated bike-category profile** (CdA ki range) + rider-specific correction, jo fill-ups se seekha jaaye.

Personal pattern: kai rides par acceleration aggressiveness, idle time aur speed distribution seekho. Inka fuel par asar usi rider ke fill-to-fill residuals se calibrate karo.

Uncertainty dikhao:

- Parameters (CdA, Crr, η, wind error) par Monte Carlo chalao aur P10–P90 range dikhao: "Est. 2.1 L (1.8–2.5 L)".
- Calibration kam ho, yaani 3 se kam fill-ups, toh "low confidence".

Wording (exactly aapki line): "This is an estimated fuel-use calculation based on bike, route, weather, load, and riding data. It is not ECU or fuel-flow sensor measurement. Actual consumption may differ."

Wind data ki sachai:

- Weather models ki wind 10 m height ki hoti hai. Sadak par rider ke level ki wind usse kam hoti hai, aur ped, ghaat ya buildings ke saath badalti hai.
- Isliye wind ka effect bhi range mein dikhao.

Mic wind analysis: future research, abhi build mat karo.

- **Kyun mushkil hai:** mic noise ka level phone, mount, case, helmet, engine, exhaust aur rain ke saath badalta hai. Isse wind ki direction nahi milti, sirf turbulence milti hai.
- **Validation protocol pehle chahiye:**
  - still-air din par GPS speed ke against mic level ki curve banao;
  - phir handheld anemometer ke saath crosswind runs karo.
  - Agar same phone aur mount par bhi spread bada aaye, toh feature band.
- **Agar banao toh Android aur privacy ki shartein:**
  - `RECORD_AUDIO` runtime permission.
  - Foreground service type `microphone`, jo app visible hone par hi start ho sakti hai.
  - Android 12+ par mic indicator dikhega.
  - Mic sirf user-started ride mein aur > 15 km/h steady speed par chale. Kam speed par band.
  - Sirf derived features (band energies) rakho.
- **Honest privacy line:** "Audio sirf phone par wind analysis ke liye process hota hai; raw audio save ya upload nahi hota." **"Speech kabhi capture nahi hoti" mat bolo.** Mic speech sunta hai, bas aap use store nahi karte.

OBD/ECU:

- BS6-era bikes mein on-board diagnostics hai.
- Lekin standard port/Bluetooth adapter se fuel-rate milna har two-wheeler par guaranteed nahi hai. Connectors aur protocols aksar manufacturer-specific hote hain.
- Jahan reliable fuel-rate mile, use ground truth ki tarah use karke model calibrate karo. Har bike par assume mat karo.

---

12. Open-data and licence checklist

Har source ke liye `data_sources` table mein yeh likho:

- licence;
- attribution text;
- commercial use allowed ya nahi;
- update method;
- fetch date;
- version.

Legal advice lekar confirm karo, khaas kar API business se pehle.

Source	Licence / terms (verify current)	Use	Caveat
OpenStreetMap (Geofabrik India extracts, daily)	**ODbL 1.0**, "© OpenStreetMap contributors"	Roads, map-matching, POIs, `maxspeed`, `highway=speed_camera`	**Share-alike:** OSM se derived database public use/API mein doge toh woh bhi ODbL. Hazard DB ko apne coordinates par rakho aur OSMF ki community guidelines (horizontal layers / collective database) padhke legal opinion lo.
Overture Maps transportation	ODbL (OSM-derived)	Road graph alternative	Wahi share-alike
OGD India (data.gov.in)	Government Open Data License – India (GODL), commercial use attribution ke saath	Dataset-by-dataset	Har dataset ki freshness aur coordinates check karo
Copernicus DEM GLO-30/90	Free incl. commercial, attribution	Elevation/slope	Repo pehle GLO-90 use kar chuka hai
SRTM	Public domain	Elevation backup	—
Bhuvan / NRSC (CartoDEM etc.)	Bhuvan/NRSC ke apne terms, registration	Low-lying analysis	Commercial reuse terms **likhit mein** confirm karo. National Geospatial Policy 2022 ne Indian entities ke liye bahut kuch liberalise kiya hai, phir bhi har dataset ki shart padho.
IMD	API access registration/whitelisting par	Nowcast, warnings	Commercial redistribution rights clear nahi; IMD se poochho
Open-Meteo	Data CC BY 4.0; free API sirf non-commercial	Weather	Commercial ke liye paid plan
MoRTH blackspots	Reports/PDF, clean public coordinate list nahi mili (repo ke `data/master/blackspots/README.md` mein documented)	Blackspot layer	Officially maango; scraping nahi
Municipal open data	City-wise	Roadworks/closures	Har city alag; coordinates aur update rate check karo

Rules jo kabhi nahi tootne chahiye:

- Google Maps ya Street View scraping nahi.
- Kisi aur ka camera/CCTV feed bina licence nahi.

Contributor licence (API business ke liye zaroori):

- Sign-up terms mein rider aapko apni reports use, process aur sublicense karne ka haq de.
- Ya phir aap community data ko ODbL ke under open release karo, aur API mein service bechein, data nahi.
- Yeh faisla launch se pehle karo.

---

13. Privacy and safety design

- **Device par hi rehta hai:** poora trace, raw IMU, ghar/kaam ki jagah, aur ride history. Retention setting ho, e.g. 90 din, aur ek tap mein "delete all".
- **Public mein jaata hai:** sirf woh hazards jo rider ne chune. Har hazard ke saath:
  - point / segment;
  - direction;
  - class;
  - severity;
  - coarse time ("3 ghante pehle").
- **Private zones:** start/end ke 250 m (repo mein pehle se hai). Ise ghar/office ke saved places tak badhao.

Sparse roads ka khatra (brutal honest point):

- Gaon ki sadak par sirf ek rider ne report kiya aur uska nickname dikh raha hai, toh poore gaon ko pata chal jaayega woh kab wahan tha.
- **Isliye:**
  - hazard details par nickname **mat dikhao**; sirf counts aur badge level ("ek Pakka Scout ne report kiya");
  - publication mein random 1–6 ghante ki delay do;
  - kam traffic wale segments par "x ghante pehle" ki jagah "aaj" ya "kal" jaisa coarse time dikhao.

Identity trade-off:

- Badges aur unique-rider counting ke liye **stable pseudonymous contributor ID** chahiye. Repo ki daily-pseudonym design isme kaam nahi karti.
- **Do modes do:**
  - *Anonymous sensor sharing:* daily pseudonym; sirf sensor evidence aur smooth passes jaate hain.
  - *Contributor account:* stable ID, badges aur reviews. Opt-in, phone OTP ya email.

Law:

- India ka **DPDP Act 2023** aur uske notified Rules location ko personal data maante hain.
- Purpose-wise consent, deletion request aur grievance contact chahiye.
- Current obligations lawyer se confirm karo.

Abuse defences:

- Rate limits.
- Account age aur trust ke hisaab se alert eligibility.
- Passage attestation.
- Impossible-travel checks (ek account 2 ghante mein 400 km door ke hazards).
- Ek network address se bahut saari accounts dikhe toh flag.
- Moderation queue.
- Play Integrity API: free hai, lekin Google service hai. Aapki choice. Isse modified apps mushkil hote hain, band nahi.
- **Honest limit:** ek coordinated group fake accounts se ek jagah ko confirm kar sakta hai. Defence yeh hai:
  - trust-weighted alert eligibility;
  - human moderation;
  - "disputed" state.

Rider safety:

- Live screen par interaction nahi.
- Review sirf ruk kar.
- Koi gamification jo ride ke dauraan poke kare, woh nahi.

---

14. 12-month roadmap (3–5 log ki team)

Months	Focus	Deliverables	Gate
1–2	Detector foundation	Two-fix interpolation, peak-time stamping, ring buffer, rule-based 6-class v0, mount-moved rejection, `DatasetRecorder` ground-truth rides (team: 3+ bikes, 2+ phones, 500+ km), labelling tool	Localisation p50/p90 measured on surveyed spots
3–4	Review flow + v1 model	Post-ride review UI, publish per hazard, GBDT v1 (group split), Room ride store, deletion	Detector precision/recall per class reported by bike/phone
4–6	Community backend	PostGIS, road segments from OSM extract, merge/dedupe, reviews, agreement/state machine, offline corridor cache, speed-aware alerts	Closed pilot (50–100 riders, one district, e.g. Koraput–Araku or Rourkela–Cuttack)
6–8	Trust & identity	Contributor accounts (opt-in), reputation, first 3 badges, abuse checks, moderation	Gaming test (internal red team), privacy review
8–10	Ride Story + fuel ranges	Offline video/poster, masking, fuel estimate with P10–P90, honest wording	Share rate, fuel-estimate error vs fill-ups
10–12	Scale + API groundwork	1D-CNN v2 (if data allows), speed limits (OSM where tagged), licence review, API keys/metering (internal only)	Launch gates (§15) met in pilot district

Mic wind, OBD, LLM aur predictions: in 12 months mein sirf research spikes, product nahi.

---

15. Five biggest risks and how to test them

1. **Cold start (data nahi, toh value nahi):**
   - **Test:** ek corridor par dense pilot. Measure karo: kitne % km par last 7 din mein ≥ 2 riders ki smooth pass ya report hai.
   - Agar 20–30% bhi nahi, toh launch district chhota karo.
   - Pehle din se value smooth passes ("ridden, nothing felt") aur aapki team ki ground-truth rides se aati hai.
2. **False positives se trust tootna:**

    - **Test:** "false voice alerts per 100 km" pilot mein napo. Rider ka "galat alert" tap button rakho.
    - Is metric ka target launch se pehle tay karo aur mano.
3. **Privacy leak (sparse roads, nicknames):**
    - **Test:** red team kuch public reports se ek pilot rider ka ghar ya routine dhoondhne ki koshish kare. Agar ho jaaye, toh design fail.
4. **Distraction / safety liability:**
    - **Test:** pilot mein alert ke baad ki hard-braking events (DynamicsCore deta hai) count karo.
    - Agar alerts ke baad sudden braking badh rahi hai, toh wording aur distance badlo.
5. **Licence / legal (ODbL share-alike, Bhuvan/IMD terms, contributor rights):**
    - **Test:** API business se pehle legal memo lo. Har dataset ki `commercial_ok` field "verified in writing" na ho toh feature off.

Launch gates (team ke targets, claims nahi):

- per-class precision/recall on group-split test set;
- location p90;
- false alerts per 100 km;
- review completion rate;
- independent audit: confirmed hazards mein se kitne audit ride mein sach mein mile;
- battery %/hour;
- crash-free sessions.

---

16. Claims: safe vs avoid

Bol sakte ho:

- "Recent riders ke reports aur phone sensors se road hazards dikhata hai, kab aur kitne riders ne confirm kiya, uske saath."
- "8 of 10 reviewing riders confirmed this hazard (last 30 days)." Sirf jab yeh sach mein reviews ka count ho.
- "Ridden by 3 riders in the last 7 days with no bumps felt." (Phone paani ya bajri feel nahi kar sakta, yeh saath mein bolo.)
- "Condition unknown." Jab data nahi.
- "Estimated fuel use: 1.8–2.5 L. It is not ECU or fuel-flow sensor measurement."
- "Your route stays on your phone; only hazards you choose to publish are shared."
- "Speed limit 60 km/h (from OpenStreetMap)." Jahan tagged ho.

Kabhi mat bolo:

- "X% probability the pothole is there" / "92% accurate".
- "Detects all potholes" / "road is safe" / "prevents accidents".
- "Exact fuel consumption".
- "Mic never captures speech".
- "Complete speed-limit / camera data" / "avoid challans".
- "Verified by the government". Jab tak official source na ho, aur tab bhi source ka naam lo.
- "AI predicted waterlogging" bina "prediction, not a report" ke.
- Kuch bhi VAHAN/mParivahan se, jab tak authorised integration na ho.

Number plate / VIN / engine number (Feature 6), short mein:

- **Model / year / variant:** haan, ride aur fuel model ke liye.
- **Number plate:** optional, sirf phone par encrypted. Kaam:
    - offline emergency card ("is bike ka owner/emergency contact");
    - user khud insurance aur PUC expiry dates daale toh reminders. In reminders ke liye plate zaroori bhi nahi.

  Public stories, reports ya server par plate kabhi nahi. - **VIN / chassis / engine number:** mat lo, jab tak kisi manufacturer ke saath recall/service integration na ho. Bina wajah ka sensitive data bas liability hai.

Seedhi priority

1. Two-fix event-time localisation aur 6-class v0 fix karo.
2. Phir post-ride review UI banao.
3. Phir explicit-review agreement layer, current sensor-evidence layer ke saath.
4. Card se "confidence %" hatao.
5. Deep Search ko local-only karo.

Yeh paanchon repo mein kuch hafton ka kaam hain, aur aapki trust story inhi par khadi hai.